

Aadhaar Holder

An Android wallet for Aadhaar in the ISO/IEC 18013-5 mobile-document (mdoc) format. It provisions a resident's Aadhaar as an mdoc and presents it to a verifier either as a **zero-knowledge proof** (proving a requested fact without disclosing the underlying data) or as an **ISO 18013-5 selective disclosure** (sharing the actual signed attribute values). Both modes operate online (over the internet through a relay) and offline (Bluetooth Low Energy and NFC).

The verifier counterpart is the **Aadhaar Verifier** application. The two applications are delivered as independent repositories and share no code module. The complete wire protocol between them is specified in [docs/HOLDER_VERIFIER_PROTOCOL.md](#).

- **Package:** `com.mdochoholder.app`
 - **Minimum SDK:** 29 · **Target SDK:** 34 · **Compile SDK:** 36
 - **Language / UI:** Kotlin, Jetpack Compose (Material 3)
 - **Zero-knowledge engine:** Google longfellow-zk via OpenWallet Foundation Multipaz
-

Contents

1. [Overview](#)
 2. [Capabilities](#)
 3. [Architecture](#)
 4. [Repository structure](#)
 5. [User interface](#)
 6. [Design system](#)
 7. [Activities](#)
 8. [Credential provisioning and the device key](#)
 9. [Presentation flows](#)
 10. [Build](#)
 11. [Configuration](#)
 12. [Permissions](#)
 13. [Dependencies](#)
 14. [Security](#)
 15. [Documentation](#)
 16. [Terminology](#)
-

1. Overview

The application provisions a signed Aadhaar mdoc from UIDAI and binds it to a hardware-backed device key in the Android Keystore. The resident's credential is rendered as an Aadhaar card, and can be presented in two modes:

- **Zero-knowledge proof.** The holder discloses the requested attribute values (for example, `age_above18 = "Yes"`) together with a proof that they are genuinely signed by the issuer, without transmitting the issuer's signature over the MSO and without revealing any other attribute. The verifier is cryptographically assured of authenticity and holder binding.
- **Selective disclosure (ISO 18013-5).** The holder transmits the actual values of the approved attributes together with the issuer-signed Mobile Security Object (MSO), for verifiers that require the real data.

Each mode is available over the internet and in offline, in-person settings.

Online and offline presentation

Each presentation is delivered over one of three transports. They differ only in how the data travels between the two devices; the request, the proof, and the verification are identical in all three.

- **Online** — the two devices do not talk to each other directly. They exchange the request and the proof through a small store-and-forward server on the internet (the "relay"): the verifier uploads the request, this app downloads it, produces the proof, and uploads it, and the verifier downloads it. The devices may be on different networks and any distance apart, provided both have internet. The verifier's QR contains an HTTPS web address (the relay).

- **Offline (Bluetooth)** — this app connects directly to the verifier's device over Bluetooth and sends the proof device-to-device, with no server and no internet. The two phones must be close together. The verifier's QR contains the verifier's Bluetooth key rather than a web address.
- **Offline (NFC tap)** — the same as Bluetooth, except the connection is started by tapping the two phones together instead of scanning a QR. NFC only carries the small engagement; the proof is then transferred over Bluetooth.

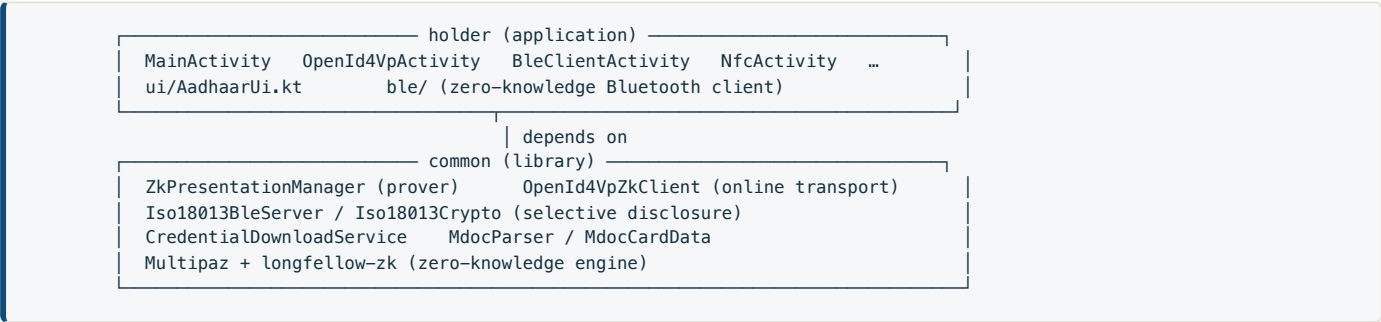
2. Capabilities

Presentation mode	Transport	Internet	Disclosed
Zero-knowledge proof	OpenID4VP over HTTPS (relay)	Yes	Requested attributes; issuer signature withheld
Zero-knowledge proof	OpenID4VP over Bluetooth LE	No	Requested attributes; issuer signature withheld
Zero-knowledge proof	NFC engagement to Bluetooth LE	No	Requested attributes; issuer signature withheld
Selective disclosure	ISO 18013-5 over Bluetooth LE (QR or NFC engagement)	No	Requested attributes and issuer signature (MSO)

3. Architecture

The project comprises two Gradle modules:

- **holder** — the application (`com.mdochoolder.app`): activities, Compose UI, NFC host-card emulation, and the zero-knowledge Bluetooth client.
- **common** — a library (`com.example.mdoc.common`): credential download, the zero-knowledge prover, the online OpenID4VP client, the ISO 18013-5 selective-disclosure server, and mdoc parsing. This module encapsulates the reusable presentation logic.



The native longfellow prover ships for the `arm64-v8a` and `x86_64` ABIs only; these are set in the application module's `abiFilters` .

4. Repository structure

```
.
├── settings.gradle.kts          # includes :holder and :common
├── build.gradle.kts            # top-level plugins and versions
├── gradle.properties
├── gradlew · gradlew.bat · gradle/ # Gradle wrapper
├── docs/
│   ├── HOLDER_VERIFIER_PROTOCOL.md # complete wire protocol
│   ├── HOLDER_VERIFIER_PROTOCOL.pdf
│   └── Holder-README.pdf           # PDF render of this README
├── holder/                     # application module (com.mdochoholder.app)
│   └── src/main/
│       ├── AndroidManifest.xml
│       ├── java/com/example/mdoc/holder/
│       │   ├── MainActivity.kt      # home, share, settings; provisioning; ISO 18013-5 presentation
│       │   ├── OpenId4VpActivity.kt # online zero-knowledge presentation (OpenID4VP)
│       │   ├── BleClientActivity.kt # offline zero-knowledge presentation over Bluetooth LE
│       │   ├── ZkScanActivity.kt    # QR scanner
│       │   ├── NfcActivity.kt       # NFC reader-mode engagement
│       │   ├── NdefService.kt       # HCE service for ISO 18013-5 handover
│       │   ├── ble/
│       │   │   └── ui/AadhaarUi.kt  # zero-knowledge Bluetooth client (protocol, crypto)
│       │   └── ui/AadhaarUi.kt      # design system and digital card
│       ├── com/mdochoholder/app/nfc/MdocHostApuService.kt
│       └── res/                     # drawables, Manrope fonts, Lottie, launcher icons
└── common/                      # library module (com.example.mdoc.common)
    └── src/main/java/com/example/mdoc/common/
        ├── CredentialDownloadService.kt # UIDAI provisioning and device-key generation
        ├── ZkPresentationManager.kt     # zero-knowledge prover (longfellow)
        ├── OpenId4VpZkClient.kt        # online OpenID4VP client
        ├── Iso18013BleServer.kt · Iso18013Crypto.kt # selective disclosure over Bluetooth LE
        ├── MdocParser.kt · MdocCardData.kt # mdoc parsing and card field mapping
        ├── MdocVerifier.kt             # issuer certificate-chain verification
        └── NfcHelper.kt · NdefHandoverBuilder.kt
```

5. User interface

The interface is implemented entirely in Jetpack Compose (Material 3) and styled to match the UIDAI visual identity. The primary surfaces are defined in `MainActivity.kt` and `ui/AadhaarUi.kt`.

Navigation

```
Provisioning (no credential present)
└── CredentialSourceScreen – UID entry, download

Main application (bottom navigation: Home · Share · Settings)
├── Home      – Aadhaar card (flip: front = photo, name, DOB, gender; back = address)
│             and access to the full attribute list
├── Share     – Zero-Knowledge Proof → Scan verifier QR · Tap verifier
│             Selective Share       → Show QR for reader · Tap reader
└── Settings – issuer certificate verification, raw credential, re-provisioning
```

Surfaces

Surface	Location	Description
CredentialSourceScreen	MainActivity.kt	UID entry and credential download, with progress and error states
HolderApp	MainActivity.kt	Bottom-navigation shell (AadhaarBottomNav)
HomeTab	MainActivity.kt	Aadhaar card and full-details entry point
ShareTab	MainActivity.kt	Zero-knowledge and selective-disclosure actions
SettingsTab	MainActivity.kt	Certificate verification, raw credential, re-provisioning
FieldsDisplay	MainActivity.kt	Full attribute list with rendered photograph
BleClientActivity	BleClientActivity.kt	Consent, determinate progress, success animation
OpenId4VpActivity	OpenId4VpActivity.kt	Online consent and success

Aadhaar card

DigitalAadhaarCard (in ui/AadhaarUi.kt) is a flippable card driven by graphicsLayer(rotationY) :

- **Front** (CardFront): a 118 dp square photograph, the resident name, date of birth formatted as DD-Mon-YYYY , and gender, over the Aadhaar card artwork (h_back.webp).
- **Back** (CardBack): resident name, address, and regional-language address, left-aligned.

Card fields are supplied by MdocCardData . The card, colours, and typography are the primary points of visual customisation.

Controls and actions

Every interactive control and the action it triggers.

Provisioning screen (CredentialSourceScreen).

Control	Action
UID field	Accepts a 12-digit UID; enables Download Aadhaar when 12 digits are entered
Download Aadhaar	Calls downloadCredential(uid) : fetches the mdoc from UIDAI (CredentialDownloadService), generates the device key in the Keystore, stores the mdoc to filesDir , extracts the card fields, and switches to the Home tab. On failure it clears any partial key material and shows the error

Home tab.

Control	Action
Aadhaar card	Tap, horizontal swipe, or the < > arrows flip the card between the front (photo, name, DOB, gender) and the back (addresses). No network or navigation
Share with a verifier	Navigates to the Share tab
View full details	Opens the full attribute list (FieldsDisplay)

Share tab — Zero-Knowledge Proof section.

Control	Action
Scan verifier QR	Launches the QR scanner (ZkScanActivity). If the scanned text begins with OPENID4VP://connect , it starts BleClientActivity (offline Bluetooth) with the engagement as EXTRA_URI ; otherwise it starts OpenId4VpActivity (online) with the scanned URL
Tap verifier	Enables NFC reader mode (enableZkNfcReader). On tapping the verifier it reads the tag over ISO-DEP (SELECT AID F04D444F4356 , SELECT NDEF, READ), extracts the OPENID4VP://connect URL, and starts BleClientActivity with it

Share tab — Selective Share section.

Control	Action
Show QR for reader	Calls <code>startQrPlusBlePresentation</code> : loads the device key, builds the ISO 18013-5 Device Engagement, starts the Bluetooth server advertising (<code>Iso18013BleServer</code>), and shows the device-engagement QR for a reader to scan
Tap reader	Calls <code>startNfcPlusBlePresentation</code> : builds the ISO 18013-5 NDEF handover, configures the HCE service (<code>NdefService</code>), and starts the Bluetooth server; the reader taps to engage, then the transfer proceeds over Bluetooth

Settings tab.

Control	Action
Verify issuer certificate (toggle)	Enables the issuer-certificate verification panel
Run verification	Runs <code>Iso18013Crypto.verifyIacaCertChain(mdoc, iaca)</code> and shows a pass/fail result with the IACA and Document Signer subjects
Show raw Aadhaar (Base64)	Opens <code>RawCredentialDialog</code> with the Base64 mdoc
Switch / re-download Aadhaar	Returns to the provisioning screen to enter a new UID

Presentation activities.

Control	Screen	Action
Share	<code>BleClientActivity</code> (consent)	Generates the ZK proof for the provable attributes (<code>ZkPresentationManager.generateZkDocument</code>), wraps it as a <code>DeviceResponse</code> , and streams it to the verifier over the encrypted Bluetooth channel; the screen shows determinate progress, then the result
Share	<code>OpenId4VpActivity</code> (consent)	Generates the ZK proof and POSTs the <code>vp_token</code> to the verifier's <code>response_uri</code> (<code>OpenId4VpZkClient.presentZk</code>); the screen shows progress, then the result
Cancel	consent screens	Closes the activity and aborts the presentation
Done	result screens	Closes the activity
Close	Selective-Share dialog	Stops the Bluetooth server and dismisses the QR

In each consent screen, attributes that exceed the circuit size limits are shown as unavailable and are excluded from the proof; **Share** is enabled only when at least one attribute is provable.

6. Design system

All visual constants and reusable composables are centralised in `holder/.../ui/AadhaarUi.kt` .

Colour palette (`Aadhaar` object)

Token	Hex	Use
<code>Navy</code>	<code>#083459</code>	Primary actions, headers, primary text
<code>NavyDark</code>	<code>#010F2A</code>	Header gradient
<code>Bg</code>	<code>#FAF4EE</code>	Screen background
<code>Surface</code>	<code>#FFFFFF</code>	Cards and surfaces
<code>Ink / InkStrong</code>	<code>#23313C / #010F2A</code>	Body / heading text
<code>Muted</code>	<code>#757575</code>	Secondary text
<code>Success / Green</code>	<code>#2EAE7D</code>	Success states
<code>Red</code>	<code>#DA251D</code>	Error states
<code>Saffron</code>	<code>#FDB913</code>	Accent
<code>CardBrown / CardBrownDark / CardGold</code>	<code>#726044 / #5E4C30 / #C4A26D</code>	Card text and gradient

Typography

The Manrope family (`res/font/manrope_*`) in five weights. `AadhaarTheme` applies Manrope as the default text style without imposing a text colour, allowing explicit colours on coloured surfaces.

Reusable composables

Composable	Purpose
<code>AadhaarTheme</code>	Material theme wrapper with the Manrope default
<code>BrandTopBar</code>	Header with the Aadhaar emblem and tagline
<code>AadhaarEmblem</code>	The Aadhaar emblem
<code>DigitalAadhaarCard</code>	The flippable Aadhaar card
<code>PrimaryButton</code> / <code>SaffronButton</code>	Full-width primary actions
<code>OutlineActionButton</code>	Secondary action
<code>ActionListItem</code>	Clickable list row with badge, title, subtitle
<code>SectionLabel</code>	Section header
<code>LottieSuccess</code>	Success animation (<code>R.raw.success_anim</code>)
<code>ShareSection</code> (in <code>MainActivity.kt</code>)	Container for a group of share actions

Assets

`ic_aadhaar_splash.xml` (emblem), `ic_aadhaar_logo.png` (wordmark), `h_back.webp` (card artwork), `success_anim.json` (Lottie), the Manrope fonts, and the adaptive launcher icons under `res/mipmap-anydpi-v26/`.

7. Activities

Registered in `holder/src/main/AndroidManifest.xml`.

Activity	Launch mode	Entry contract	Role
<code>MainActivity</code>	singleTask	Launcher; handles NFC <code>NDEF</code> / <code>TAG_DISCOVERED</code>	Home, provisioning, selective-disclosure presentation
<code>OpenId4VpActivity</code>	singleTask	<code>openid4vp://</code> , <code>haip://</code> , HTTPS; extra <code>EXTRA_URI = "openid4vp_uri"</code>	Online zero-knowledge presentation
<code>BleClientActivity</code>	singleTask	Extra <code>EXTRA_URI = "openid4vp_ble_uri"</code>	Offline zero-knowledge presentation (Bluetooth central)
<code>ZkScanActivity</code>	—	Started for result	QR scanner for verifier engagement
<code>NdefService</code> (service)	—	<code>HOST_APDU_SERVICE</code>	ISO 18013-5 NFC handover
<code>MdocHostApduService</code> (service)	—	<code>HOST_APDU_SERVICE</code>	Selective-disclosure NFC AID

Inter-activity navigation from `MainActivity` :

```
Intent(this, BleClientActivity::class.java).putExtra(BleClientActivity.EXTRA_URI, engagement)
Intent(this, OpenId4VpActivity::class.java).putExtra(OpenId4VpActivity.EXTRA_URI, uri)
```

How a scanned engagement is routed. When the user scans the verifier's QR (or reads its NFC tag), `MainActivity` obtains a short "engagement" string but does not perform the presentation itself — it starts the appropriate presentation activity and passes that string to it. Android activities receive data through *intent extras* (named key/value pairs), and `EXTRA_URI` is the key under which the engagement string is passed:

- `BleClientActivity.EXTRA_URI = "openid4vp_ble_uri"` — carries the offline Bluetooth engagement `OPENID4VP://connect?name=...&key=<X25519 public key>`.

- `OpenId4VpActivity.EXTRA_URI = "openid4vp_uri"` — carries the online engagement `openid4vp:///client_id=...&request_uri=...` .

`MainActivity` chooses the target activity from the engagement's form: a string beginning with `OPENID4VP://connect` is an offline Bluetooth engagement and is routed to `BleClientActivity` ; any other `openid4vp:///HTTPS` engagement is online and is routed to `OpenId4VpActivity` . The receiving activity reads the value with `intent.getStringExtra(EXTRA_URI)` and acts on it — for the Bluetooth path it parses the verifier's X25519 key and connects; for the online path it fetches the request from the `request_uri` . `EXTRA_URI` is therefore the mechanism that tells the presentation activity which verifier and session to connect to.

8. Credential provisioning and the device key

Implemented in `common/CredentialDownloadService.kt` .

- **Source.** UIDAI Pehchaan staging endpoint `https://pehchaanstage.uidai.gov.in/csm/api/v1` .
- **Flow.** `getNonce` → generate an EC P-256 key in the Android Keystore (alias `mdoc_device_key`) with an attestation challenge derived from the nonce → `getCredential` → decrypt the returned JWE (RSA-OAEP-256 with A256GCM) → extract the base64 mdoc.
- **Storage.** The mdoc is written to `filesDir/aadhaar_mdoc_b64.txt` ; the device private key remains in the Keystore.
- **Key and credential consistency.** The device key is generated as part of provisioning, and its public half is embedded in the issuer-signed MSO. No credential is bundled with the application. Re-provisioning overwrites the key atomically, keeping the credential and key synchronised.

The stored mdoc is a CBOR map containing `issuerSigned` (namespaces and the `issuerAuth` COSE_Sign1 over the MSO) and `docType = in.gov.uidai.aadhaar.1` . `MdocParser` and `MdocCardData` map elements to the card fields (name, date of birth, gender, address, and the resident photograph).

9. Presentation flows

Field-level detail is specified in `docs/HOLDER_VERIFIER_PROTOCOL.md`.

Action (Share)	Transport	Activity	Disclosed
Scan verifier QR (zero-knowledge)	Bluetooth LE, or online for an HTTPS engagement	<code>BleClientActivity</code> / <code>OpenId4VpActivity</code>	Requested attributes; issuer signature withheld
Tap verifier (zero-knowledge)	NFC engagement to Bluetooth LE	<code>NfcActivity</code> → <code>BleClientActivity</code>	Requested attributes; issuer signature withheld
Show QR for reader (selective)	ISO 18013-5 over Bluetooth LE	<code>MainActivity</code>	Requested attributes and MSO
Tap reader (selective)	ISO 18013-5 (NFC handover) to Bluetooth LE	<code>MainActivity</code> / <code>NdefService</code>	Requested attributes and MSO

The zero-knowledge presentation proceeds as follows:

1. Obtain the OpenID4VP request (scanned or deep-linked online, or read over Bluetooth offline). The request is an OpenID4VP query in DCQL form naming the document type, the claims, and the circuit.
2. Present the consent screen. Attributes exceeding the circuit size limits (element value above 64 bytes or `IssuerSignedItem` above 119 bytes) are marked unavailable for selection.
3. Compute the session transcript for the session.
4. Generate the proof via `ZkPresentationManager.generateZkDocument(...)` . Holder binding is included as a device signature over the transcript.
5. Wrap the proof in an ISO 18013-5 `DeviceResponse` , encode it as a `vp_token` , and deliver it — by HTTP POST online, or chunked over the encrypted Bluetooth channel offline.

Code path (online): `OpenId4VpZkClient.fetchRequest(qr)` fetches the **signed** request, verifies its ES256 signature and **pins** the verifier certificate (`VerifierRequestAuth`), then parses it → `ZkPresentationManager.checkSizes(...)` gates oversized attributes → `OpenId4VpZkClient.presentZk(...)` computes the transcript, calls `ZkPresentationManager.generateZkDocument(...)` (which selects the circuit by attribute count, filters the credential to the requested attributes, signs for holder binding, and runs the prover), wraps the result with `buildZkDeviceResponse(...)` , base64url-encodes it into the `vp_token` , and posts it — **encrypted** as a `direct_post.jwt` JWE (ECDH-ES / A256GCM) when the request asks for encryption. The same `ZkPresentationManager` calls are used offline by `BleClientActivity` . The proving steps are documented with the actual code and an actual request and response in the protocol specification, section 13.

10. Build

Requirements: JDK 17; Android SDK 36; a target device or emulator with the `arm64-v8a` or `x86_64` ABI (required by the longfellow native library).

The zero-knowledge engine is native (compiled C/C++) code, which runs only on the CPU architecture (Android **ABI**) it was compiled for. The `multipaz-longfellow` dependency provides the compiled library (`libzkp.so`) for `arm64-v8a` (64-bit ARM) and `x86_64` (64-bit Intel/AMD) only, and the application module sets `ndk { abiFilters = ["arm64-v8a", "x86_64"] }` accordingly. The application therefore runs on 64-bit ARM devices and `x86_64` emulators.

```
./gradlew :holder:assembleDebug      # build the debug APK
./gradlew :holder:installDebug       # install on a connected device
# artifact: holder/build/outputs/apk/debug/holder-debug.apk
```

For headless or CI builds, set `JAVA_HOME` to a JDK 17 and provide `local.properties` with `sdk.dir=/path/to/Android/sdk` (this file is not tracked).

11. Configuration

Setting	Location	Notes
UIDAI endpoint	<code>common/CredentialDownloadService.kt</code>	Staging URL
Document type	<code>in.gov.uidai.aadhaar.1</code>	Resolved from the credential
Application id and name	<code>holder/build.gradle.kts</code> , <code>res/values/strings.xml</code>	<code>com.mdochoholder.app</code>
Theme, colours, typography	<code>holder/.../ui/AadhaarUi.kt</code>	Single source of truth

The online transport requires no configuration here: the holder follows the `request_uri` and `response_uri` supplied in each verifier request.

12. Permissions

Declared in `holder/src/main/AndroidManifest.xml` :

- Bluetooth (`BLUETOOTH_CONNECT` , `BLUETOOTH_SCAN` , `BLUETOOTH_ADVERTISE`) for the offline transports, requested at runtime on Android 12 and above.
- NFC for engagement.
- Internet for provisioning and the online transport.
- Camera for QR scanning.

The manifest also registers the two host-card-emulation services with their AID filters under `res/xml/` .

13. Dependencies

Principal libraries (`holder/build.gradle.kts` , `common/build.gradle.kts`):

- Multipaz: `multipaz` , `multipaz-longfellow` , `multipaz-doctypes` , `multipaz-compose` — the mdoc and zero-knowledge stack.
- Jetpack Compose, Material 3, and Lottie for the interface.
- ZXing for QR generation and scanning.
- BouncyCastle for X25519, HKDF, and ECDSA.
- CBOR (`co.nstant.in:cbor`), Gson, OkHttp, and Nimbus JOSE+JWT.

The toolchain uses Kotlin 2.2, as required by `multipaz-longfellow` , with the Kotlin Compose plugin.

14. Security

- **Holder binding.** Every presentation is signed by the hardware-backed device key; a copied credential file is unusable without the device.
- **Data minimisation.** In zero-knowledge mode, only the requested predicate or claims leave the device; attribute values, other attributes, and the issuer signature are never transmitted.
- **Replay resistance.** Each proof is bound to a per-session transcript.
- **Offline confidentiality.** The Bluetooth channel uses AES-256-GCM following an X25519 handshake.
- **No embedded secrets.** No credential or key material is shipped in the APK; the credential is provisioned per device. Signing keys and `local.properties` are excluded from version control.

15. Documentation

- [docs/HOLDER_VERIFIER_PROTOCOL.md](#) — the complete holder-verifier wire protocol, with real request and response captures and the zero-knowledge engine inputs and outputs.

Standards: ISO/IEC 18013-5 and 18013-7, OpenID4VP 1.0, OAuth JAR (RFC 9101), the W3C Digital Credentials API, Google longfellow-zk, and OpenWallet Foundation Multipaz.

16. Terminology

Term	Definition
mdoc	ISO 18013-5 mobile document (CBOR)
MSO	Mobile Security Object; the issuer-signed structure committing to attribute digests and the device key
IssuerSigned	The attribute items and issuer signature within the mdoc
Device key	Per-device key establishing holder binding
OpenID4VP	The presentation request and response protocol
DCQL	The query language carried in an OpenID4VP request
vp_token	The response object carrying the DeviceResponse
DeviceResponse	The ISO 18013-5 response container; its <code>zkDocuments</code> entry carries the zero-knowledge proof
Circuit	A longfellow circuit compiled for a fixed attribute count
Selective disclosure	Disclosing actual attribute values rather than a proof
Relay	The store-and-forward mailbox used by the online transport